ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΣΗΜΑΤΩΝ ΕΛΕΓΧΟΥ ΚΑΙ ΡΟΜΠΟΤΙΚΗΣ

# ΣΧΕΔΙΑΣΗ ΣΥΣΤΗΜΑΤΩΝ ΑΥΤΟΜΑΤΟΥ ΕΛΕΓΧΟΥ

## ΕΛΕΓΧΟΣ ΤΑΧΥΤΗΤΑΣ ΚΑΙ ΘΕΣΗΣ ΜΕ ΕΛΕΓΚΤΕΣ PI, PD και PID

ΑΓΓΕΛΟΣ ΚΑΜΑΡΙΑΔΗΣ

# ΜΕΡΟΣ 1

Χρησιμοποιούμε την εξής απλή γραμμική περιγραφή για το σύστημα ελέγχου της ταχύτητας ενός αυτοκινήτου:

$$\dot{v} = -\alpha v + bu + d,$$

όπου: v είναι η ταχύτητα του αυτοκινήτου, u η επιτάχυνση (ή η επιβράδυνση) του, α και b είναι θετικές σταθερές συσχετιζόμενες με την τριβή, τη μάζα και την ισχύς της μηχανής του αυτοκινήτου ενώ d είναι μία εξωτερική διαταραχή που εφαρμόζεται στο όχημα, λόγω της κλίσης του δρόμου στον οποίο κινείται.

Οι ονομαστικές τιμές του α και του b είναι: α=b=1.

**<u>1.1 Υπολογισμός της συνάρτησης μεταφοράς G, από την είσοδο ελέγχου στην έξοδο</u>**

Θεωρώντας έναν αναλογικό ελεγκτή, με αρχική διαταραχή d=0 και α=b=1, υπολογίζουμε την συνάρτηση μεταφοράς με την εξής μέθοδο:

$$sV(s) = -\alpha V(s) + bU(s) + D(s), \text{ u(0)=0}$$

$$\Longrightarrow V(s) = \frac{b}{s+a}U(s) + \frac{1}{s+a}D(s)$$

$$\Longrightarrow G_u(s) = \frac{V(s)}{U(s)} = \frac{1}{s+1}, \text{ εφόσον α=b=1 και d=0}$$

Άρα η συνάρτηση μεταφοράς από την είσοδο ελέγχου στην έξοδο, είναι η: $\boldsymbol{G_u(s) = \frac{1}{s+1}}$

**<u>1.2 Υπολογισμός της συνάρτησης μεταφοράς G, από την διαταραχή στην έξοδο</u>**

$$V(s) = \frac{b}{s+a}U(s) + \frac{1}{s+a}D(s)$$

$$\Longrightarrow G_d(s) = \frac{V(s)}{U(s)} = \frac{1}{s+1}, \text{ εφόσον u(0)=0}$$

Άρα η συνάρτηση μεταφοράς από την είσοδο ελέγχου στην έξοδο, είναι η: $\boldsymbol{G_d(s) = \frac{1}{s+1}}$

**<u>2. Εύρεση βηματικών αποκρίσεων</u>**

Δημιουργούμε ένα σύστημα κλειστού βρόχου αποτελούμενο από έναν αναλογικό ελεγκτή, ούτως ώστε να αναλύσουμε πως το κέρδος K, επηρεάζει την απόκριση και το steady state error.

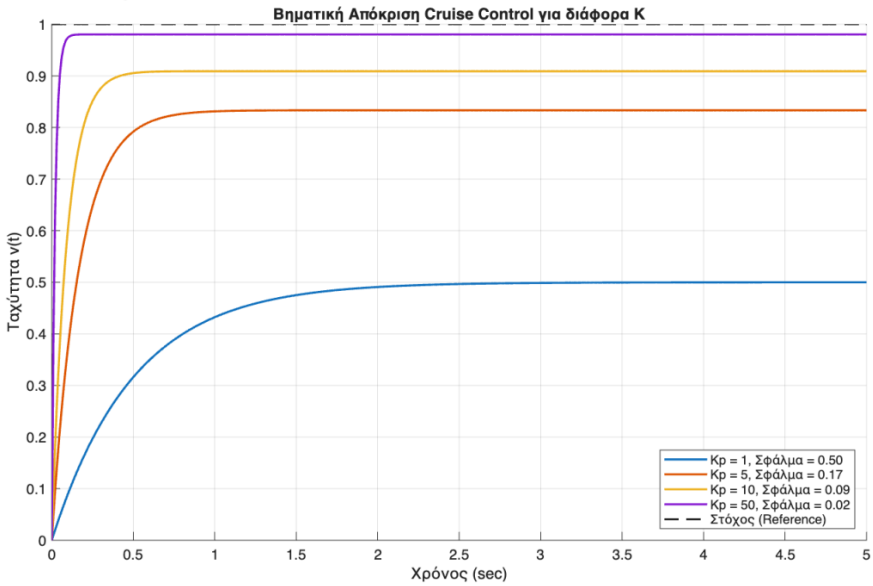
Η συνάρτηση μεταφοράς του προς έλεγχο συστήματος είναι: $\boldsymbol{G(s) = \frac{1}{s+1}}$
Η συνάρτηση μεταφοράς του ελεγκτή είναι: C(s) = K

Άρα η συνάρτηση μεταφοράς κλειστού βρόχου θα είναι: $\boldsymbol{T(s)} = \frac{\boldsymbol{C(s)G(s)}}{\boldsymbol{1+C(s)G(s)}} = \frac{\boldsymbol{K}}{\boldsymbol{1+s+K}}$

Με τη χρήση του MATLAB, υπολογίστηκαν οι βηματικές αποκρίσεις για τις εξής τιμές κέρδους: K = 1, 5, 10 , 50

```
>> num_plant = [1];
>> den_plant = [1 1];
>> G = tf(num_plant, den_plant);
>> K_values = [1, 5, 10, 50];
>> t = 0:0.01:5;
>> figure;
>>
>> hold on;
>> grid on;
>>
>> for k = K_values
C = k;
T = feedback(C * G, 1);
[y, t_out] = step(T, t);

error = 1 / (1 + k);
plot(t_out, y, 'LineWidth', 1.5, ...
'DisplayName', sprintf('Kp = %d, Σφάλμα = %.2f', k, error));
end
>> y_ref = ones(size(t));
>> plot(t, y_ref, 'k--', 'LineWidth', 1.2, 'DisplayName', 'Στόχος (Reference)');
>> title('Βηματική Απόκριση Cruise Control για διάφορα K');
>> xlabel('Χρόνος (sec)');
>> ylabel('Ταχύτητα v(t)');
>> legend('Location', 'southeast');
>> hold off;
```




$$e_{ss} = 1 - \lim_{s\to 0} sT(s)\frac{1}{s} \Longrightarrow e_{ss} = \frac{1}{1+K} \Longrightarrow e_{ss} \longrightarrow 0 \ \kappa\alpha\theta\acute{\omega}\varsigma \ K \longrightarrow \infty$$

Παρατηρούμε ότι καθώς το K αυξάνεται, το steady state error μειώνεται. Ωστόσο, ποτέ δεν μηδενίζεται τελείως με έναν απλό αναλογικό ελεγκτή, διότι το άπειρο (ή το πολύ μεγάλο) κέρδος δεν είναι πρακτικά εφικτό σε συστήματα 1ης τάξης.

## 3. Σχεδιάση PI Controller

Με έναν απλό αναλογικό ελεγκτή (P), παρουσιάζεται πάντα ένα σφάλμα. Αυτό συμβαίνει γιατί ο ελεγκτής χρειάζεται σφάλμα για να παράγει σήμα εισόδου. Για να εξαλείψουμε το σφάλμα, προσθέτουμε τον ολοκληρωτικό όρο (I). Άρα μεταβαίνουμε στη χρήση ενός PI ελεγτκή. Ο ολοκληρωτής αθροίζει το σφάλμα με την πάροδο του χρόνου. Όσο υπάρχει

σφάλμα, η έξοδος του ολοκληρωτή αυξάνεται, αναγκάζοντας το σύστημα να διορθώσει μέχρι το σφάλμα να γίνει ακριβώς μηδέν.

Η συνάρτηση μεταφοράς κλειστού βρόχου του PI controller θα είναι: $\boldsymbol{C_{PI}(s)} = \frac{\boldsymbol{K_p s + K_i}}{\boldsymbol{s}}$, ενώ η συνάρτηση μεταφοράς ανοιχτού βρόχου του θα είναι: $\boldsymbol{L(s)} = \frac{\boldsymbol{K_p s + K_i}}{\boldsymbol{s(s+1)}}$

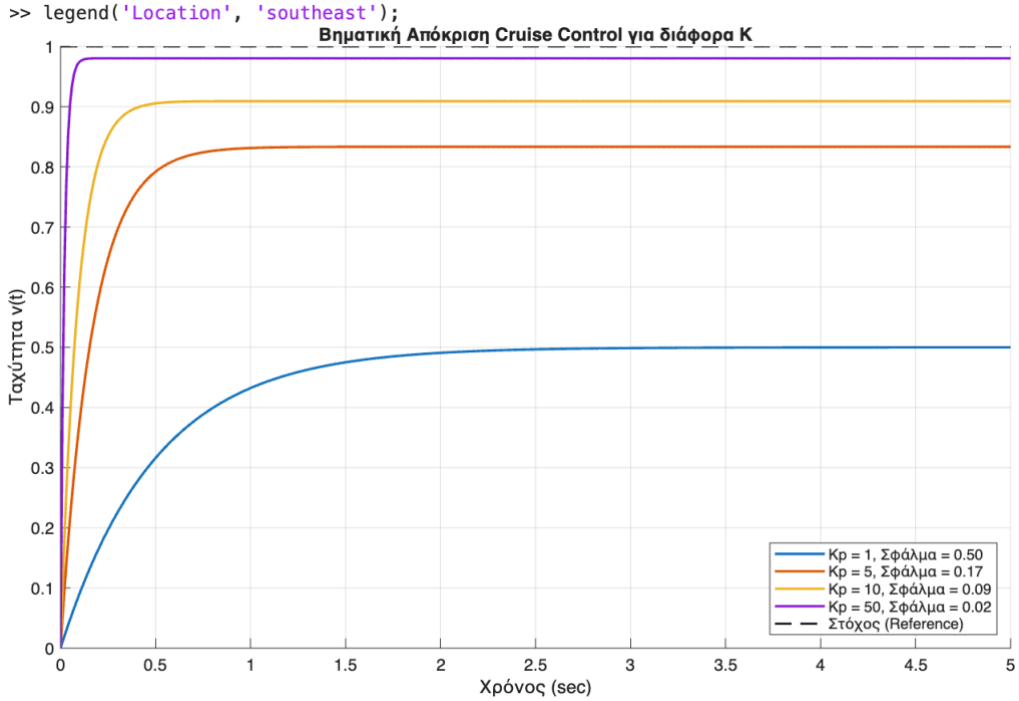
Επιλέγουμε τις τιμές των $K_p$ και $K_i$ με τρόπο ώστε να εξασφαλίζεται η ευστάθεια.

$$1 + L(s) = 0 \Longrightarrow s^2 + (1 + K_p)s + K_i = 0$$

Για να εξασφαλιστεί η ευστάθεια, $1 + K_p > 0$ και $K_i > 0$

Επιλέγουμε τις εξής τιμές: $K_p$=$K_i$=1

```
>> G = tf([1], [1 1]);
>> Kp_only = 5;
>> C_p = Kp_only;
>> T_p = feedback(C_p * G, 1);
>> Kp = 1;
>> Ki = 1;
>> C_pi = pid(Kp, Ki);
>> T_pi = feedback(C_pi * G, 1);
>> t = 0:0.01:10;
>> [y_p, t_p] = step(T_p, t);
>> [y_pi, t_pi] = step(T_pi, t);
>>
>> figure;
>> hold on;
>> grid on;
>>
>> y_ref = ones(size(t));
>> plot(t, y_ref, 'k--', 'LineWidth', 1.5, 'DisplayName', 'Στόχος (Reference = 1)');
>> plot(t_p, y_p, 'r-.', 'LineWidth', 1.5, 'DisplayName', 'P-Controller (Kp=5)');
>> plot(t_pi, y_pi, 'b', 'LineWidth', 2, 'DisplayName', 'PI-Controller (Kp=1, Ki=1)');
>> title('Σύγκριση P vs PI: Εξάλειψη Σφάλματος');
>> xlabel('Χρόνος (sec)');
>> ylabel('Ταχύτητα');
>> legend('Location', 'southeast');
```



## 4. Γιατί δεν χρησιμοποιούμε μεγάλο κέρδος;

Διότι οι ατέλειες του πραγματικού κόσμου, όπως καθυστερήσεις και δυναμικές που αγνοήσαμε αρχικά, θα ωθήσουν το σύστημα προς την αστάθεια.

Για να πετύχουμε $e_{ss} < 0.5\% \Longrightarrow \frac{1}{1+K_p} < 0{,}005 \Longrightarrow K_p + 1 > 200 \Longrightarrow K > 199$

Οπότε, επιλέγουμε: $K_p = 200$.

Εφόσον η δυναμική του actuator περιγράφεται από την συνάρτηση μεταφοράς:

$$G_a(s) = \frac{1}{(1 + \frac{s}{100})^2}$$

Παρατηρούμε πως παρουσιάζονται δύο επιπλέον πόλοι στο s = -100. Έτσι, όταν το σύστημα προσπαθεί να κινηθεί πολύ γρήγορα, αυτοί οι πόλοι προσθέτουν καθυστέρηση φάσης που οδηγεί σε αστάθεια.

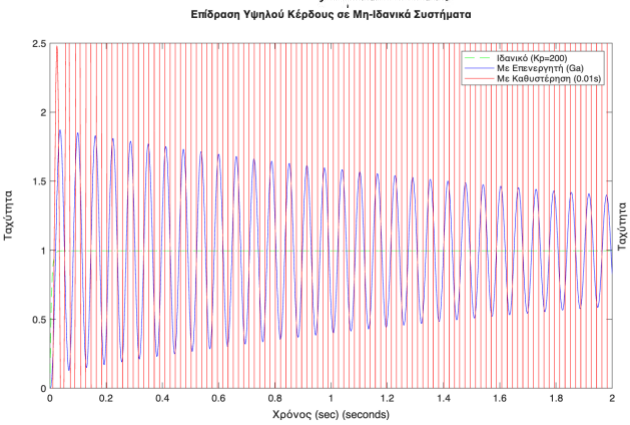
Στα παρακάτω γραφήματα, παρουσιάζονται οι ταλαντώσεις που εισάγει η δυναμική του actuator σε σύγκριση με την ασταθή συμπεριφορά του συστήματος, όταν αντί για τη δυναμική του actuator, εφαρμόζουμε απλά μία καθυστέρηση 0,01 sec.

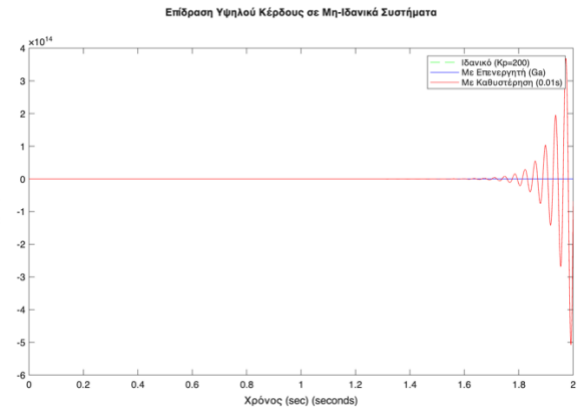
```
>> s = tf('s');
G = 1 / (s + 1);
Kp = 200;
Ga = 1 / (1 + s/100)^2;
L_actuator = Kp * Ga * G;
T_actuator = feedback(L_actuator, 1);
G_delay = G;
G_delay.InputDelay = 0.01;
L_delay = Kp * G_delay;

% Κλειστός Βρόχος
T_delay = feedback(L_delay, 1);
t = 0:0.001:2;

figure;
hold on; grid on;

T_ideal = feedback(Kp * G, 1);
step(T_ideal, t, 'g--');
step(T_actuator, t, 'b');
step(T_delay, t, 'r');
legend('Ιδανικό (Kp=200)', ...
       'Με Επενεργητή (Ga)', ...
       'Με Καθυστέρηση (0.01s)');
title('Επίδραση Υψηλού Κέρδους σε Μη-Ιδανικά Συστήματα');
xlabel('Χρόνος (sec)');
ylabel('Ταχύτητα');
ylim([0 2.5]);
```



## 5. Γιατί δεν χρησιμοποιούμε απλά έναν feedforward όρο;

***(A)*** Στόχος του Feedforward όρου, είναι η πρόβλεψη της απαιτούμενης εισόδου, ούτως ώστε να πετύχουμε την επιθυμητή έξοδο.

Για την πρόβλεψη χρησιμοποιούμε την εξίσωση ισορροπίας: $\dot{v} = 0$. Για τις ονομαστικές τιμές του συστήματος, η εξίσωση ισορροπίας είναι:

$$0 = -1v + 1u \Longrightarrow \mathrm{u} = v$$

$$\Longrightarrow \text{για δεδομένη έξοδο } \mathrm{y} = \mathrm{r} \Longrightarrow v = r \Longrightarrow \mathrm{u} = r$$

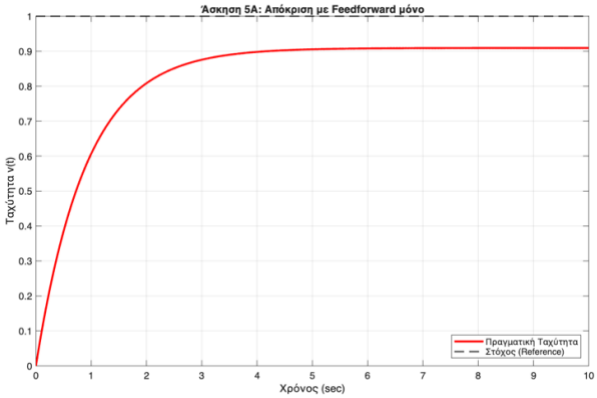
$$\text{εφόσον } \mathrm{u} = \mathrm{k_r r} \text{ (από τη δομή του ελεγκτή)} \Longrightarrow \mathrm{k_r} = 1$$

<u>Τριβή:</u> Αυξημένη κατά 10% ⟹ α' = 1,1
<u>Ισχύς:</u> Μειωμένη κατά 10% ⟹ b' = 0,9
<u>Διαταραχή:</u> Το 10% της ονομαστικής τιμής ⟹ d' = 0,1

Χρησιμοποιώντας τις παραπάνω τιμές, από την εξίσωση ισορροπίας: $\dot{v} = -\alpha v + bu + d$, υπολογίζουμε την απόκριση, δηλαδή την ταχύτητα του οχήματος, η οποία ισούται με:

$$\mathrm{v} \approx 0{,}909$$

```
>> a_nom = 1;
>> b_nom = 1;
>> r = 1;
>> kr = a_nom / b_nom;
>> fprintf('Υπολογισμένο κέρδος Feedforward (kr): %.2f\n', kr);
Υπολογισμένο κέρδος Feedforward (kr): 1.00
>> a_real = 1.1;
>> b_real = 0.9;
>> d_disturbance = 0.10 * (kr * r);
>> u_control = kr * r;
>> total_input = (b_real * u_control) + d_disturbance;
>> s = tf('s');
>> G_sys = 1 / (s + a_real);
>> t = 0:0.01:10;
>> [v, t_out] = step(total_input * G_sys, t);
>>
>> figure;
>> plot(t_out, v, 'r', 'LineWidth', 2);
>> hold on;
>> yline(r, 'k--', 'LineWidth', 1.5);
>> grid on;
>>
>> title('Άσκηση 5Α: Απόκριση με Feedforward μόνο');
>> xlabel('Χρόνος (sec)');
>> ylabel('Ταχύτητα v(t)');
>> legend('Πραγματική Ταχύτητα', 'Στόχος (Reference)', 'Location', 'SouthEast');
>> final_v = v(end);
>> error = r - final_v;
```

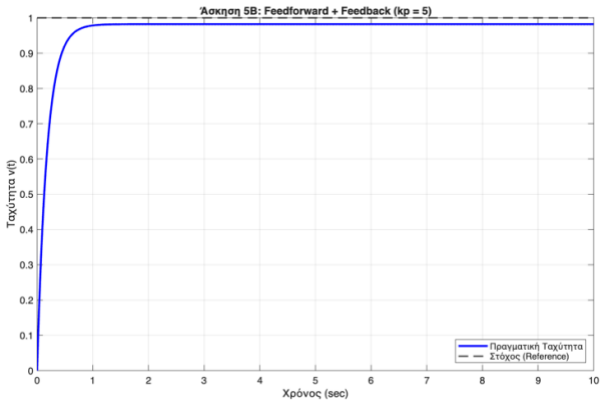



Από το παραπάνω γράφημα, παρατηρούμε πως στην ιδανική περίπτωση, το σύστημα συγκλίνει στην τιμή αναφοράς.

***(B)*** Τώρα, προσθέτουμε έναν αναλογικό όρο $k_p(r - y)$, ο οποίος βλέπει το σφάλμα και το διορθώνει.

$$\Longrightarrow u = k_r r + k_p(r - y)$$

```
>> a_nom = 1;
Extend MATLAB capabilities with MathWorks toolboxes, hardware support, and community submissions
>> a_real = 1.1;
>> b_real = 0.9;
>> d_disturbance = 0.10 * (kr * r);
>> kp = 5;
>> a_closed = a_real + (b_real * kp);
>> total_input_closed = (b_real * (kr + kp) * r) + d_disturbance;
>> s = tf('s');
>> G_closed = 1 / (s + a_closed);
>> t = 0:0.01:10;
>> [v_fb, t_out] = step(total_input_closed * G_closed, t);
>>
>> figure;
>> plot(t_out, v_fb, 'b', 'LineWidth', 2);
>> hold on;
>> yline(r, 'k--', 'LineWidth', 1.5);
>> grid on;
>>
>> title(['Άσκηση 5B: Feedforward + Feedback (kp = ' num2str(kp) ')']);
>> xlabel('Χρόνος (sec)');
>> ylabel('Ταχύτητα v(t)');
>> legend('Πραγματική Ταχύτητα', 'Στόχος (Reference)', 'Location', 'SouthEast');
>> final_v = v_fb(end);
>> error = r - final_v;
```




Από το παραπάνω γράφημα, παρατηρούμε πως εξακολουθεί να υπάρχει steady state error, γεγονός που οφείλεται στο ότι το $k_r$ του feedforward, αφορά το ιδανικό σύστημα, ενώ το σύστημα στο οποίο αναφερόμαστε απέχει σε τιμές α και b κατά 10%, ενώ σε διαταραχή κατά 90% από το ιδανικό σύστημα.

# ΜΕΡΟΣ 2

## 1. Συμπεριφορά συστήματος για βηματική είσοδο και αναλογικό ελεγκτή

Το σύστημα ελέγχου θέσης με το οποίο ασχολούμαστε στο 2ο μέρος, όντας 2ας τάξης, εισάγει στην ανάλυση μας, τις έννοιες υπερύψωσης , χρόνος αποκατάστασης και απόσβεση. Το σύστημα αυτό, περιγράφεται από την εξίσωση:

$$\ddot{x} = -\alpha\dot{x} + bu + d$$

όπου: x η θέση, u η είσοδος, d μια εξωτερική διαταραχή. Οι τιμές των α και b, είναι α=b=1, ενώ παράλληλα θεωρούμε μηδενική διαταραχή.

για α=b=1 και d=0, η συνάρτηση μεταφοράς του συστήματος είναι:

$$s^2X(s) = -sX(s) + U(s) \Longrightarrow X(s) = \frac{U(s)}{s^2 + s}$$

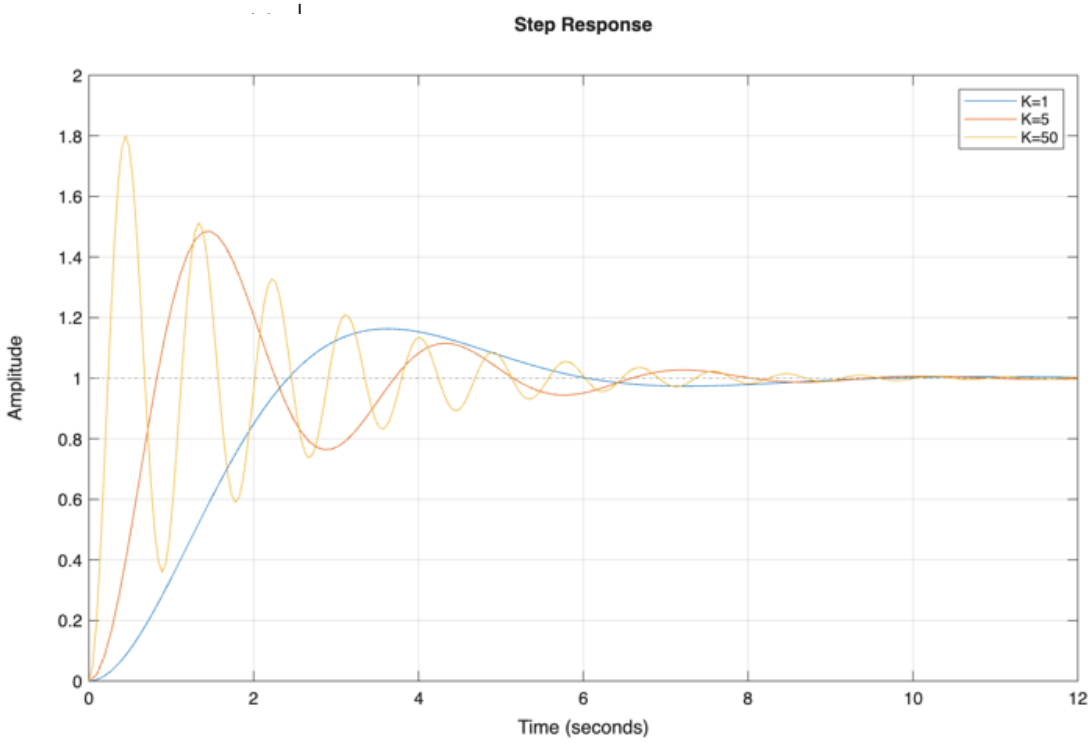
$$\Longrightarrow G(s) = \frac{X(s)}{U(s)} = \frac{1}{s(s+1)}$$

Η συνάρτηση μεταφοράς κλειστού βρόχου T(s), για τον κλειστό βρόχο που προκύπτει εισάγωντας έναν αναλογικό ελεγκτή με συνάρτηση μεταφοράς $C(s) = K_p$ είναι:

$$T(s) = \frac{K_pG(s)}{1 + K_pG(s)} \Longrightarrow T(s) = \frac{K_p}{s^2 + s + K_p}$$

Παρακάτω, φαίνεται ο κώδικας MATLAB που παράγει το γράφημα παρακάτω.

```
>> s = tf('s');
>> G = 1 / (s*(s+1));
>> K = [1, 5, 50];
>> figure;
>> hold on;
>> grid on;
>> for i = 1:length(K)
   T = feedback(K(i)*G, 1);
   step(T);
end
>> legend('K=1', 'K=5', 'K=50');
```




Από το παραπάνω γράφημα, εξάγουμε το συμπέρασμα, πως όσο και να αυξήσουμε το κέρδος, δεν μπορούμε να μειώσουμε τον χρόνο που χρειάζεται το σύστημα για να σταματήσει να ταλαντώνεται (περίπου 8 sec). Αυτό που πετυχαίνουμε αυξάνοντας το κέρδος είναι η αύξηση της συχνότητας των ταλαντώσεων. Το πλάτος της ταλάντωσης θα μειώνεται με τον ίδιο αργό ρυθμό. Επομένως, η απόκριση δεν μπορεί να γίνει ουσιαστικά γρήγορη (ευσταθής γρήγορα) μόνο με αναλογικό ελεγκτή.

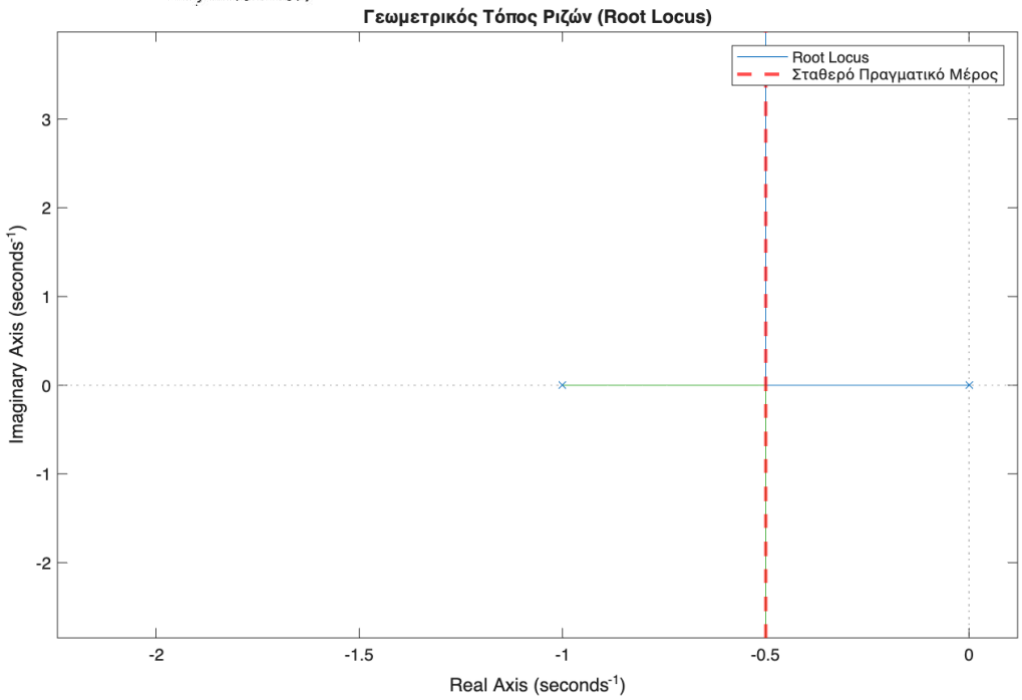
## 2. Σχεδιασμός Γεωμετρικού Τόπου Ριζών

Για τον σχεδιασμό του ΓΤΡ, χρειαζόμαστε τη συνάρτηση μεταφοράς ανοιχτού βρόχου, η οποία δίνεται από τον τύπο:

$$L(s) = \frac{K_p}{s(s+1)}$$

Παρακάτω, φαίνεται τόσο ο ΓΤΡ, όσο και ο κώδικας που τον παράγει.

```
>> s = tf('s');
>> G = 1 / (s * (s + 1));
>> figure;
>> grid on;
>> hold on;
>> rlocus(G);
>> xline(-0.5, 'r--', 'LineWidth', 2, 'DisplayName', 'Real Part = -0.5');
>> title('Γεωμετρικός Τόπος Ριζών (Root Locus)');
>> legend('Root Locus', 'Σταθερό Πραγματικό Μέρος');
>>
>> xlim([-3 1]);
>> ylim([-5 5]);
```




Οι πόλοι της συνάρτησης μεταφοράς ανοιχτού βρόχου βρίσκονται στις θέσεις:
$s_1 = 0$ και $s_2 = -1$.

Όπως ήταν αναμενόμενο, ο Γεωμετρικός Τόπος Ριζών, εξηγεί γιατί όσο και να αυξάναμε το κέρδος (ερώτημα 1), δεν μπορούσαμε να μειώσουμε τον χρόνο που χρειάζεται το σύστημα για να σταματήσει να ταλαντώνεται. Ο χρόνος αποκατάστασης, εξαρτάται αποκλειστικά από το πραγματικό μέρος των πόλων. Στον Γεωμετρικό Τόπο Ριζών αυτού του συστήματος, μόλις οι πόλοι γίνουν μιγαδικοί, το πραγματικό τους μέρος παραμένει σταθερό στο -0.5, όσο μεγάλο κέρδος και να βάλουμε. Γι' αυτό στο προηγούμενο γράφημα, όλες οι καμπύλες ηρεμούσαν στον ίδιο χρόνο. Το μόνο που άλλαζε ήταν η συχνότητα ταλάντωσης, δηλαδή, το φανταστικό μέρος των πόλων.

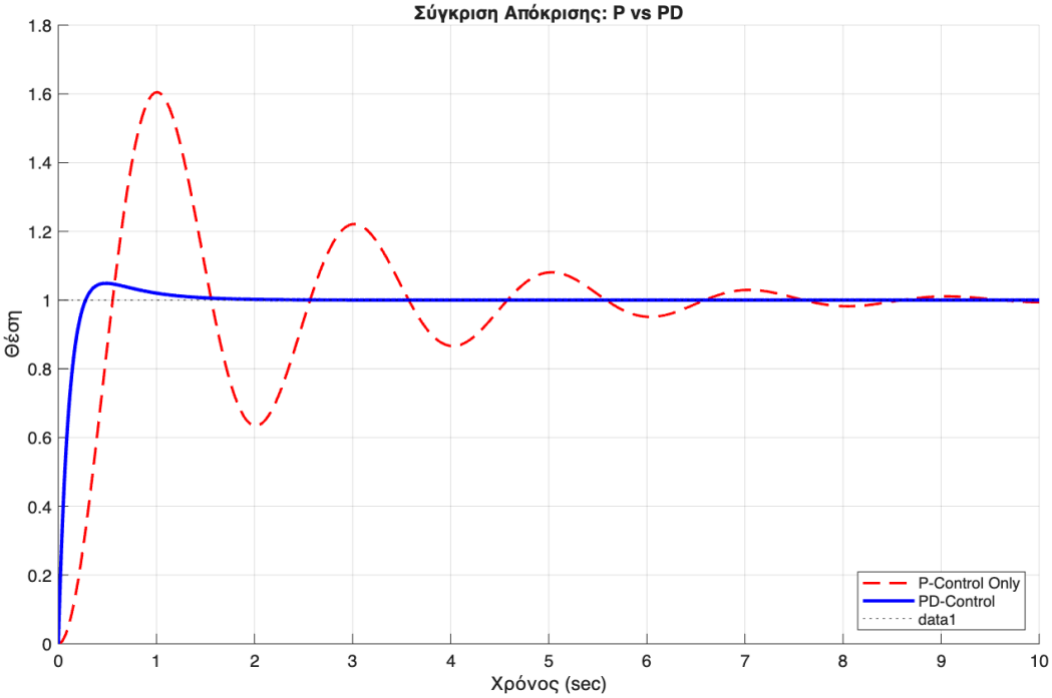
## 3. Σχεδιασμός Γεωμετρικού Τόπου Ριζών για PD Controller

Η χρήση του PD ελεγκτή, είναι η λύση στο πρόβλημα που εντοπίσαμε στα προηγούμενα ερωτήματα. Η συνάρτηση μεταφοράς του είναι:

$$C(s) = K_p + K_d s = K_d(s + \frac{K_p}{K_d})$$

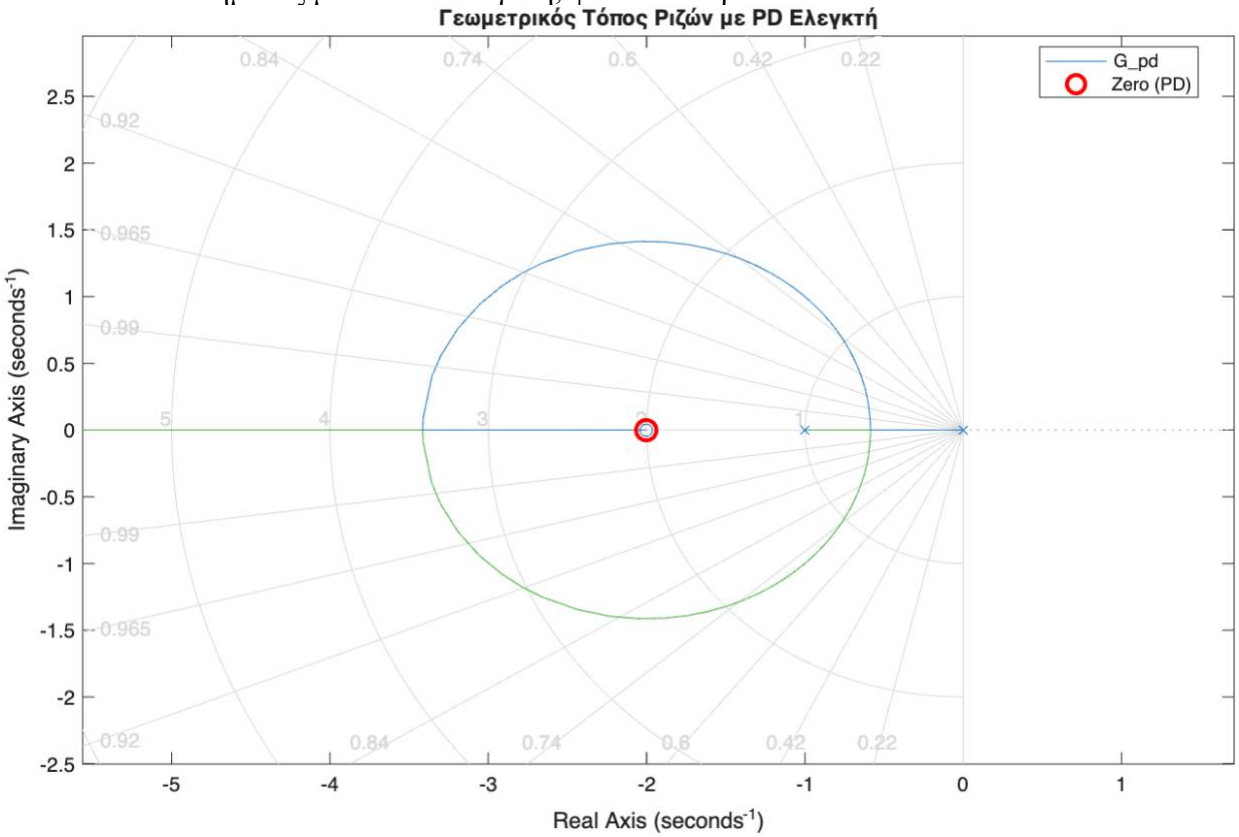
Από τη C(s), παρατηρούμε πως ο PD ελεγκτής εισάγει ένα μηδενικό στη θέση $z = - K_p/K_d$.

Παρακάτω, συγκρίνουμε τις αποκρίσεις για τις δύο περιπτώσεις.




Παρατηρούμε πως ο διαφορικός όρος (D), προβλέπει το σφάλμα και φρενάρει το σύστημα εγκαίρως, επιτρέποντας μας να αυξήσουμε το κέρδος χωρίς να προκαλέσουμε αστάθεια ή μεγάλες ταλαντώσεις.

Ο ΓΤΡ του συστήματος με τον PD ελεγκτή, φαίνεται παρακάτω.



Οι κώδικες που παράγουν τα πιο πάνω γραφήματα φάινονται παρακάτω:

```
clear;
clf;
close all;
clc;

s = tf('s');
G = 1 / (s^2 + s);
Zero_loc = 2;
C_pd_structure = s + Zero_loc;
figure;
subplot(1,2,1);
rlocus(G);
title('ΓΤΡ με P-Control (Παλιό)');
axis([-5 1 -3 3]); grid on;
xline(-0.5, 'r--', 'DisplayName', 'Όριο ταχύτητας');
subplot(1,2,2);
rlocus(C_pd_structure * G);
title('ΓΤΡ με PD-Control (Νέο)');
axis([-5 1 -3 3]); grid on;
hold on; plot(-Zero_loc, 0, 'ro', 'MarkerSize', 8, 'LineWidth', 2);
legend('Root Locus', 'Zero at -2');
figure;
hold on; grid on;
K = 10;
T_p = feedback(K * G, 1);
T_pd = feedback(K * C_pd_structure * G, 1);
t = 0:0.01:10;
[y_p, t_p]   = step(T_p, t);
[y_pd, t_pd] = step(T_pd, t);
plot(t_p, y_p, 'r--', 'LineWidth', 1.5, 'DisplayName', 'P-Control Only');
plot(t_pd, y_pd, 'b', 'LineWidth', 2, 'DisplayName', 'PD-Control');
yline(1, 'k:');
title('Σύγκριση Απόκρισης: P vs PD');
xlabel('Χρόνος (sec)');
ylabel('θέση');
legend('Location', 'Southeast');
```

```
clear;
clf;
close all;
clc;

s = tf('s');
G = 1 / (s^2 + s);
z = 2;
G_pd = G * (s + z);
figure;
rlocus(G_pd);
hold on;
grid on;
plot(-z, 0, 'r o', 'MarkerSize', 10, 'LineWidth', 2, 'DisplayName', 'Zero (PD)');
title('Γεωμετρικός Τόπος Ριζών με PD Ελεγκτή');
legend('Location', 'best');
xlim([-6 2]);
ylim([-3 3]);
```

## 4. Απόκριση για σταθερή και μη μηδενική διαταραχή - Σχεδιασμός Διαγράμματος Bode

Από τη διαφορική εξίσωση που περιγράφει το σύστημα, μεταβαίνοντας στο πεδίο Laplace παίρνουμε:

$$X(s)(s^2 + s) = U(s) + D(s)$$

Ο νόμος ελέγχου με PD ελεγκτή και αναφορά λέει: $U(s) = -C(s)X(s)$

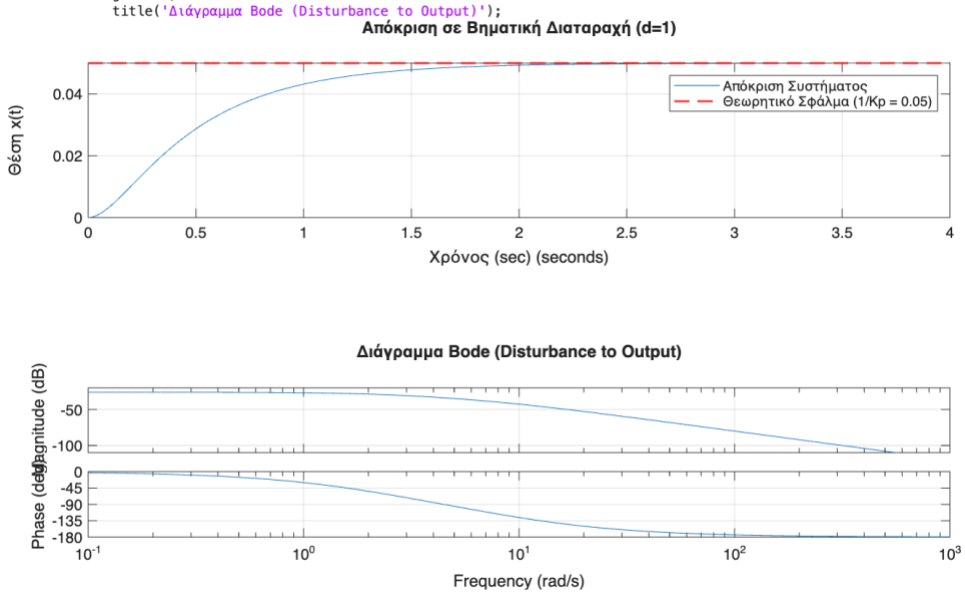
$$\Longrightarrow X(s)(s^2 + s + C(s)) = D(s)$$

Η συνάρτηση μεταφοράς από τη διαταραχή στην έξοδο είναι:

$$T_d(s) = \frac{X(s)}{D(s)} = \frac{1}{s^2 + s + C(s)}, C(s) = K_d s + K_p \Longrightarrow T_d(s) = \frac{1}{s^2 + (1 + K_d s) + K_p}$$

Παρακάτω, φαίνεται το διάγραμμα της απόκρισης σε βηματική διαταραχή και το διάγραμμα Bode, καθώς και ο κώδικας που τα παράγει.

```
>> s = tf('s');
G = 1 / (s^2 + s);
Kp = 20;
Kd = 10;
C_pd = Kd*s + Kp;
T_dist = feedback(G, C_pd);
figure;
subplot(2,1,1);
step(T_dist);
grid on;
title('Απόκριση σε Βηματική Διαταραχή (d=1)');
ylabel('Θέση x(t)');
xlabel('Χρόνος (sec)');
expected_error = 1/Kp;
yline(expected_error, 'r--', 'LineWidth', 1.5);
legend('Απόκριση Συστήματος', ['Θεωρητικό Σφάλμα (1/Kp = ' num2str(expected_error) ')']);
subplot(2,1,2);
bode(T_dist);
grid on;
title('Διάγραμμα Bode (Disturbance to Output)');
```




Για μια σταθερή διαταραχή, η τελική τιμή της εξόδου, δίνεται με τη χρήση του θεωρήματος τελικής τιμής και είναι:

$$e_{ss} = \lim_{s \longrightarrow 0} sT_d(s)\frac{1}{s} = \frac{1}{K_p}$$

Συμπερασματικά, βλέπουμε πως υπάρχει μόνιμο σφάλμα. Το σύστημα μετακινείται από τη θέση 0 και ισορροπεί στο $1/K_p$. Δεν μπορεί να μηδενίσει τη διαταραχή πλήρως, για αυτό θα χρειαζόμασταν και τον ολοκληρωτικό όρο (I).

## 5. Σχεδιασμός PID Controller

Ο PID Controller, έχει την εξής μορφή:

$$C(s) = K_p + K_d s + \frac{K_i}{s}$$

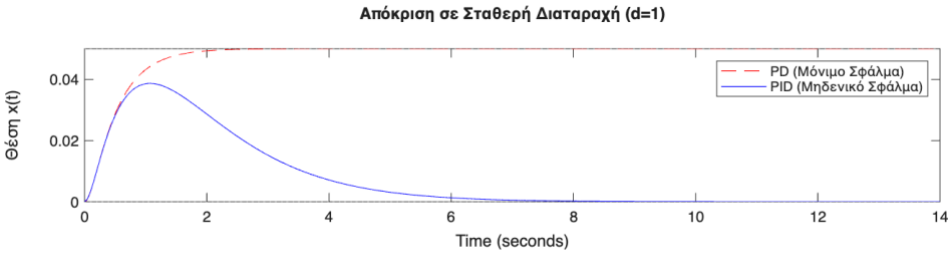
Η συνάρτηση μεταφοράς από τη διαταραχή στην έξοδο είναι:

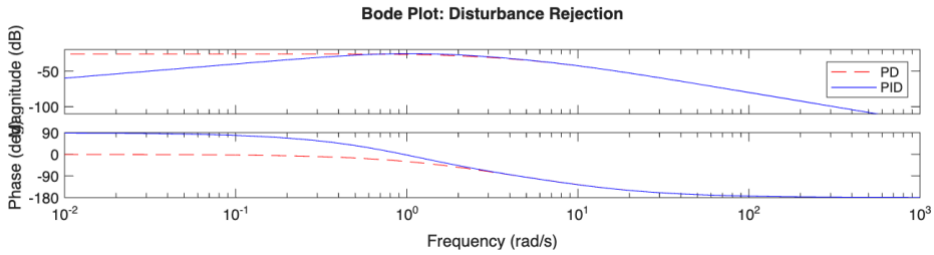
$$T_d(s) = \frac{G(s)}{1 + C(s)G(s)}$$

Επιλέγουμε τις παραμέτρους ως εξής:

- $K_p$=20 (Αναλογικό Κέρδος)
- $K_d$=10 (Διαφορικό Κέρδος)
- $K_i$=5 (Ολοκληρωτικό Κέρδος)

```
>> s = tf('s');
G = 1 / (s^2 + s);
Kp = 20;
Kd = 10;
C_pd = Kp + Kd*s;
Ki = 10;
C_pid = Kp + Kd*s + Ki/s;
T_dist_PD  = feedback(G, C_pd);
T_dist_PID = feedback(G, C_pid);
figure;
subplot(2,1,1);
hold on; grid on;
step(T_dist_PD, 'r--');
step(T_dist_PID, 'b');
legend('PD (Μόνιμο Σφάλμα)', 'PID (Μηδενικό Σφάλμα)');
title('Απόκριση σε Σταθερή Διαταραχή (d=1)');
ylabel('Θέση x(t)');
subplot(2,1,2);
hold on; grid on;
bode(T_dist_PD, 'r--');
bode(T_dist_PID, 'b');
legend('PD', 'PID');
title('Bode Plot: Disturbance Rejection');
```




Από τα παραπάνω, εξάγουμε το συμπέρασμα πως η προσθήκη του ολοκληρωτικού όρου (I) είναι απαραίτητη όταν θέλουμε να εξαλείψουμε πλήρως την επίδραση εξωτερικών, σταθερών φορτίων ή διαταραχών.

## 6. Ημιτονοειδής αναφορά

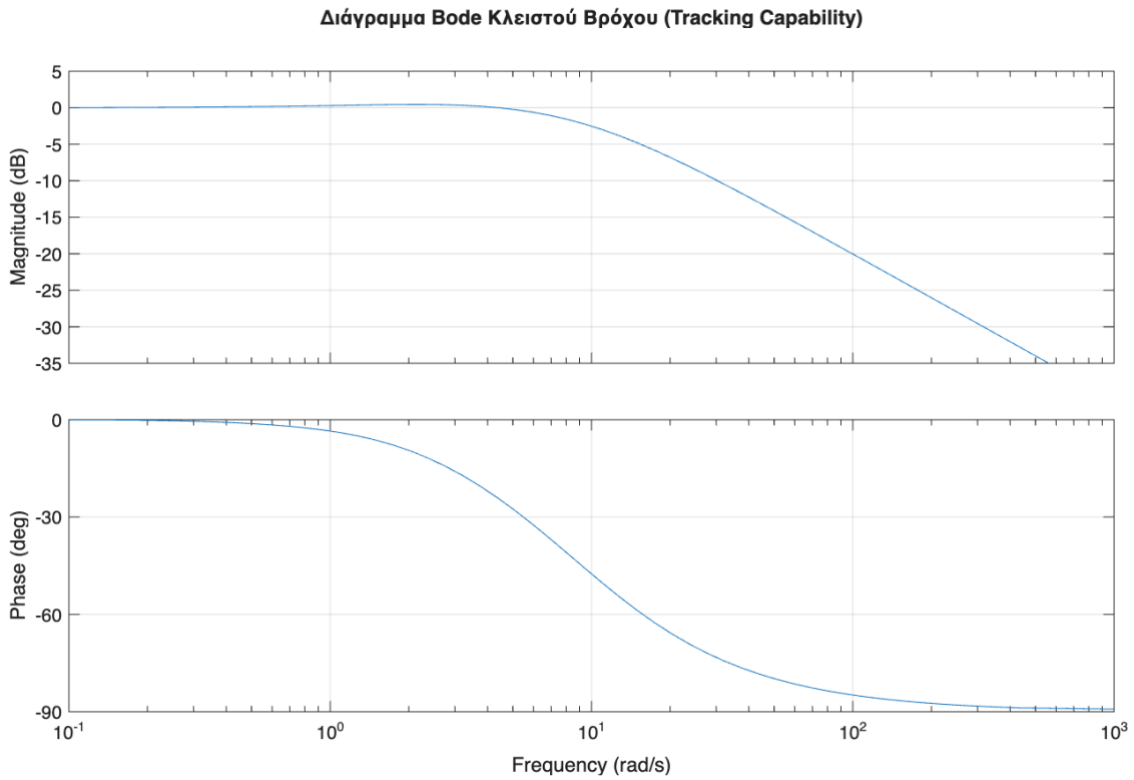
Για να δούμε πώς συμπεριφέρεται το σύστημα σε ημιτονοειδή σήματα διαφορετικών συχνοτήτων, το κατάλληλο εργαλείο είναι το διάγραμμα Bode κλειστού βρόχου από την αναφορά (r) στην έξοδο (x).

Η ικανότητα του συστήματος να παρακολουθεί την ημιτονοειδή αναφορά εξαρτάται άμεσα από τη συχνότητα του σήματος. Στις χαμηλές συχνότητες, το σύστημα εμφανίζει μοναδιαίο κέρδος (0 dB στο διάγραμμα Bode) και μηδενική διαφορά φάσης, επιτυγχάνοντας τέλεια παρακολούθηση της εισόδου.

Αντίθετα, στις υψηλές συχνότητες, παρατηρείται εξασθένηση του πλάτους (αρνητικά dB) και υστέρηση φάσης. Συνεπώς, το σύστημα αδυνατεί να ανταποκριθεί στις ταχείες μεταβολές της εντολής, με αποτέλεσμα η έξοδος να είναι εξασθενημένη και χρονικά καθυστερημένη.

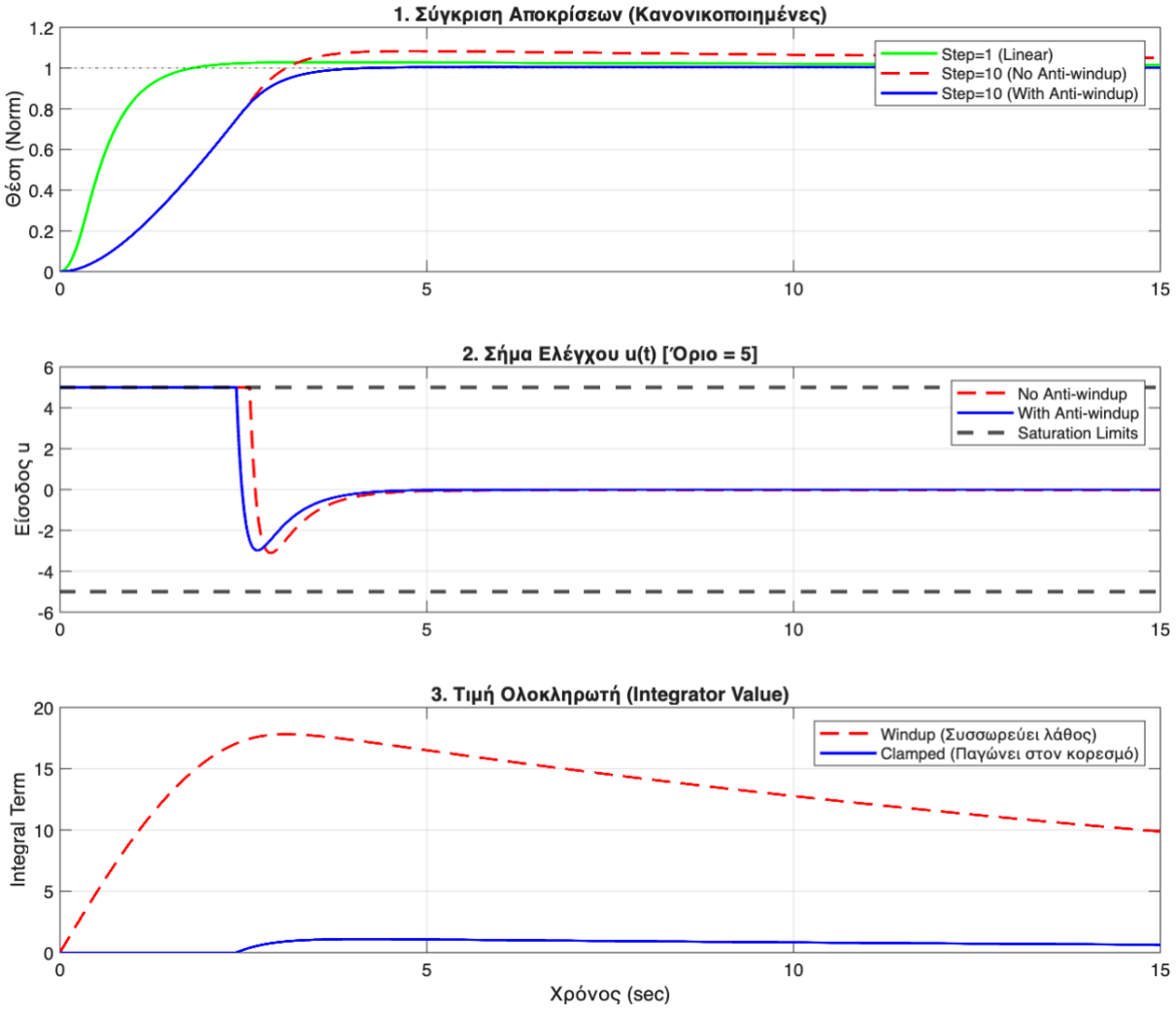
Παρακάτω, φαίνεται το διάγραμμα Bode κλειστού βρόχου και ο κώδικας που το παράγει.

```
>> s = tf('s');
G = 1 / (s^2 + s);
Kp = 20;
Kd = 10;
C_pd = Kp + Kd*s;
Ki = 10;
C_pid = Kp + Kd*s + Ki/s;
T_dist_PD  = feedback(G, C_pd);
T_dist_PID = feedback(G, C_pid);
figure;
subplot(2,1,1);
hold on; grid on;
step(T_dist_PD, 'r--');
step(T_dist_PID, 'b');
legend('PD (Μόνιμο Σφάλμα)', 'PID (Μηδενικό Σφάλμα)');
title('Απόκριση σε Σταθερή Διαταραχή (d=1)');
ylabel('θέση x(t)');
subplot(2,1,2);
hold on; grid on;
bode(T_dist_PD, 'r--');
bode(T_dist_PID, 'b');
legend('PD', 'PID');
title('Bode Plot: Disturbance Rejection');
```



## 7. Integrator Windup

Όπως προκύπτει από τα διαγράμματα, όταν το σήμα ελέγχου υπερβαίνει τα φυσικά όρια του συστήματος, οδηγώντας τον actuator σε κορεσμό, εκδηλώνεται το φαινόμενο του integrator windup. Συγκεκριμένα, παρόλο που το σφάλμα ενδέχεται να μειώνεται ή να αντιστρέφεται προσημικά, η τιμή του ολοκληρωτή παραμένει υψηλή λόγω της συσσώρευσης σφάλματος κατά το διάστημα του κορεσμού. Αυτή η κατάσταση προκαλεί έντονη υπερύψωση (overshoot) και ταλαντώσεις, καθυστερώντας την αποκατάσταση της ισορροπίας του συστήματος.



## 8. Υλοποίηση PID ελεγκτή σε C++

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

class PIDController {
public:
    PIDController(float kp, float ki, float kd, float ts, float min, float max)
        : Kp(kp), Ki(ki), Kd(kd), Ts(ts), outputMin(min), outputMax(max) {
        reset();
    }
    void reset() {
        integrator = 0.0f;
        prevError = 0.0f;
    }
    float calculate(float setpoint, float measuredValue) {
        float error = setpoint - measuredValue;
        float P = Kp * error;
        integrator += (Ki * error * Ts);
        integrator = clamp( v: integrator,  lo: outputMin,  hi: outputMax);
        float I = integrator;
        float D = Kd * (error - prevError) / Ts;
        float output = P + I + D;
        output = clamp( v: output,  lo: outputMin,  hi: outputMax);
        prevError = error;
        return output;
    }
private:
    float Kp;
    float Ki;
    float Kd;
    float Ts;
    float outputMin;
    float outputMax;
    float integrator;
    float prevError;
};
int main() {
    float Ts = 0.001f;
    PIDController motorPID( kp: 20.0f,  ki: 5.0f,  kd: 10.0f, Ts,  min: -5.0f,  max: 5.0f);
    float target_position = 10.0f;
    float current_position = 0.0f;
    cout << "Starting PID Control Simulation..." << endl;
    for (int i = 0; i < 10; i++) {
        float u = motorPID.calculate( setpoint: target_position,  measuredValue: current_position);
        current_position += u * Ts;
        cout << "Time: " << i * Ts << "s | "
                << "Pos: " << current_position << " | "
                << "Control (u): " << u << endl;
    }
    return 0;
}
```